DERMAI

AI-ASSISTED SKIN LESION IMAGE ANALYSIS

Technical Project Report and Academic Submission Dossier

A deployed research prototype combining React, FastAPI and a local EfficientNet-B0 classifier

Submitted by: _____

Roll number: _____

Faculty guide: _____

Department / Institution: _____

Submission date: 02 November 2026

Important Project Scope Statement

DermAI is an academic research prototype for demonstrating image upload, backend validation and experimental seven-class skin-lesion image classification. It is not a medical device, has not been clinically validated and must not be used to diagnose disease or guide treatment. Model scores are outputs of a classifier, not calibrated probabilities of disease.

The current implementation does not include a validated non-skin image rejection gate. An investigation using unrelated object images showed that the existing seven-class classifier can still assign a lesion label to an out-of-domain image. This limitation is documented explicitly rather than presented as a completed feature.

Report Contents

- 1. Executive Summary
- 2. Introduction and Problem Definition
- 3. Objectives and Project Scope
- 4. Requirements and Technology Stack
- 5. System Architecture and Design
- 6. Implementation Details
- 7. Dataset Preparation and Model Training
- 8. Experimental Results and Interpretation
- 9. Out-of-Domain Investigation
- 10. API Design and Deployment
- 11. Testing, Security and Privacy
- 12. Limitations and Future Work
- 13. Conclusion
- Appendix A. Faculty Demonstration Script
- Appendix B. References

1. Executive Summary

DermAI is a web-based academic research prototype designed to demonstrate a complete image-analysis workflow: a user selects a skin-lesion image, the frontend validates and previews the file, the FastAPI backend verifies the image contents, and a local EfficientNet-B0 model produces a ranked set of seven-class predictions. The application is deployed with a React + Vite frontend on Netlify and a FastAPI backend on Render.

The current build supports JPG, PNG and WEBP files up to 5 MB, a health-check endpoint, image-analysis requests, top-three prediction display and an educational disclaimer. A separate upload endpoint can persist an image and case metadata in SQLite.

Model development used ISIC 2018 Task 3 data, prepared with lesion-group-aware train/validation/test partitions. The strongest recorded run achieved validation macro-F1 of 0.7141 and exploratory test macro-F1 of 0.7157. These results require cautious interpretation because the test split was inspected across multiple experiment cycles and is not an untouched independent benchmark.

A separate out-of-domain investigation using 437 unrelated object images showed that the classifier assigned all 437 images to one of its seven lesion classes. This finding establishes a key next development priority: a separately evaluated image-suitability and non-skin rejection gate.

Project area	Current state
Web application	Deployed React + Vite frontend and FastAPI backend
Classifier	Local EfficientNet-B0 checkpoint; seven output classes; top-three scores
Model experiments	Three recorded training experiments; best recorded run summarised in this report
Dataset preparation	10,015 ISIC images; lesion-group-aware split with zero recorded group leakage
Rejection gate	Not implemented; out-of-domain investigation identifies the need
Commercial / clinical readiness	Not established; dataset rights review and independent clinical validation remain pending

2. Introduction and Problem Definition

2.1 Background

Skin-lesion image classification is a computer-vision task in which an image is mapped to one of a set of predefined categories. Deep convolutional neural networks can learn visual patterns from labelled image datasets. However, a classifier trained on a closed set of categories may still return one of those categories when it receives an unrelated image.

2.2 Problem statement

The project addresses the engineering problem of building an accessible web workflow that accepts a skin-lesion image and displays experimental classifier predictions. A second problem identified during testing is the lack of an input suitability mechanism: the existing seven-class model does not inherently know when an image is outside its training domain.

2.3 Proposed approach

DermAI combines a React-based upload interface, FastAPI endpoints, Pillow image validation and an EfficientNet-B0 classifier loaded from a backend checkpoint. A future rejection gate is planned as a separate component before the seven-class classifier. It is not part of the current working build.

3. Objectives and Project Scope

3.1 Objectives

- Build a responsive web interface for selecting, previewing and submitting image files.
- Validate file type, size and actual image contents at the backend boundary.
- Integrate a locally loaded EfficientNet-B0 checkpoint through a FastAPI inference endpoint.
- Return a consistent JSON response containing the top class, score, top-three predictions and a research disclaimer.
- Prepare a traceable training and evaluation workflow using lesion-group-aware partitions.
- Investigate out-of-domain behaviour and document requirements for a future rejection gate.

3.2 Included scope

- Frontend image upload and preview experience.
- Backend health, upload and analysis endpoints.
- Seven-class EfficientNet-B0 inference.
- Top-three prediction display and educational disclaimer.
- SQLite metadata storage for the separate `/upload` endpoint.
- Deployment and academic demonstration documentation.

3.3 Excluded from current release

- Clinical diagnosis, treatment recommendations or medical triage.
- Validated rejection of non-skin or poor-quality images.
- Prospective clinical validation or regulatory approval.
- Commercial use clearance for all training data and model weights.
- Production-grade authentication, rate limiting and full image-retention governance.

4. Requirements and Technology Stack

4.1 Functional requirements

ID	Requirement	Current implementation
FR-01	Select an image from the client device	Implemented
FR-02	Preview the selected image before analysis	Implemented
FR-03	Accept JPEG, PNG and WEBP files up to 5 MB	Frontend and backend checks
FR-04	Display backend health/connectivity status	Implemented in frontend
FR-05	Submit an image for model inference	POST /analyze
FR-06	Display top prediction and top-three alternatives	Implemented
FR-07	Display a non-diagnostic disclaimer	Implemented
FR-08	Persist uploaded file and case metadata	Separate POST /upload endpoint
FR-09	Reject unrelated non-skin images reliably	Not implemented

4.2 Non-functional requirements

- Usability: responsive interface with clear loading, result and error states.
- Input robustness: server-side format, size and image-decoding validation.
- Maintainability: separated frontend and backend projects with REST API integration.
- Portability: Python backend and browser-based frontend.

- Privacy: avoid uploading images without user permission; the separate /upload endpoint currently persists files and metadata and requires a defined retention policy before production use.
- Reliability: model inference depends on checkpoint availability and compatible PyTorch/torchvision versions.

4.3 Technology stack

Layer	Technology	Role
Frontend	React 19, Vite 8, CSS	Upload UI, image preview, status and result rendering
Backend	Python, FastAPI, Uvicorn	REST API, request validation and inference orchestration
Image processing	Pillow	Verify/decode image, convert to RGB
Deep learning	PyTorch, torchvision, EfficientNet-B0	Load checkpoint, preprocess tensor and run classification
Persistence	SQLite + filesystem	Case metadata and stored files for /upload endpoint
Deployment	Netlify + Render	Host frontend and API separately

5. System Architecture and Design

The frontend communicates with the FastAPI backend through HTTP multipart form-data. The current analysis path uses `/analyze`; the backend validates the file, preprocesses it and runs the local EfficientNet-B0 checkpoint. The response is returned as JSON and displayed by the frontend.

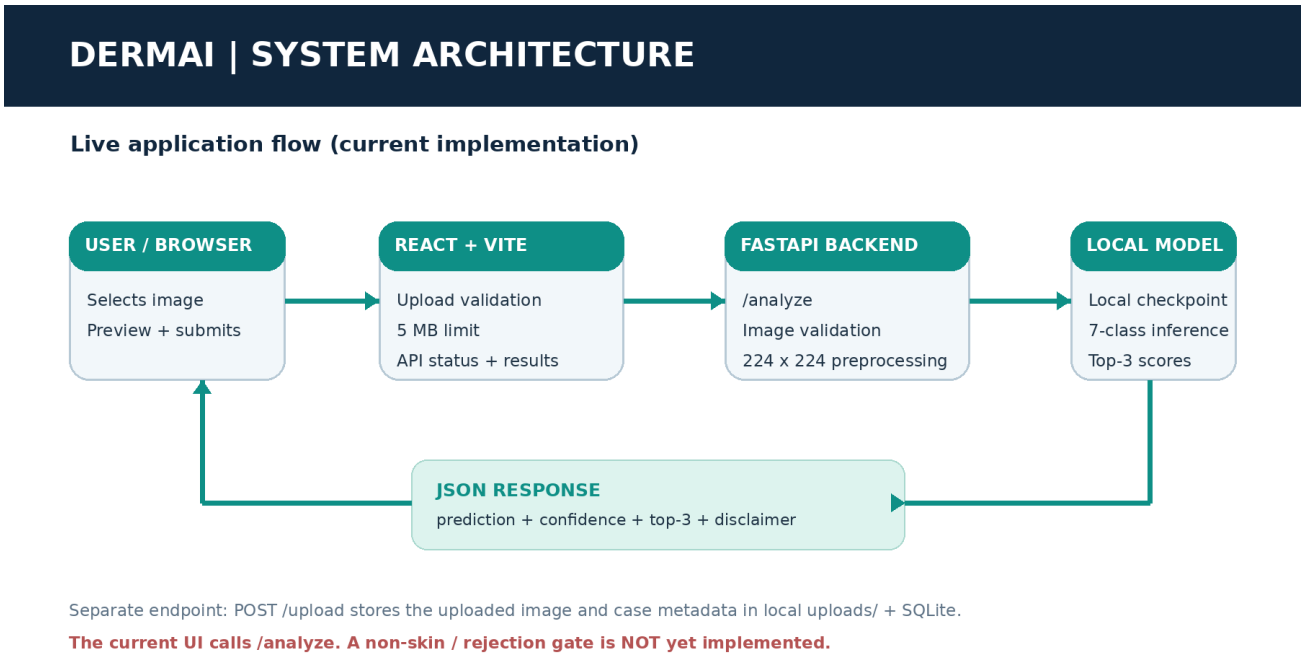


Figure 1. Current DermAI runtime architecture. The /upload persistence path is separate from the frontend analysis flow.

5.1 Frontend design

The React interface includes a research-preview header, an upload workspace, accepted file-format guidance, local preview, file removal/reset, API connection indicator, loading state, error feedback and a prediction result area. It also provides image-capture guidance and an explicit medical disclaimer.

5.2 Backend design

FastAPI exposes root and health endpoints plus separate upload and analysis operations. The application enables CORS for the configured development origins and the deployed Netlify origin. The model is loaded lazily and cached in process memory after the first analysis request.

5.3 Inference design

The analysis endpoint accepts the file in memory, checks MIME type and maximum size, verifies the actual image format using Pillow, decodes the image and converts it to RGB. The image is resized to 224 x 224 and normalized with the standard ImageNet mean and standard deviation before model inference. Softmax scores are ranked and the top three class entries are returned.

5.4 Planned rejection-gate design

A separate image-suitability component is proposed before the classifier. It should identify invalid, unsuitable or clearly out-of-domain inputs and route uncertain cases to a safe retry/review message. This component has not yet been trained, integrated or validated; it is not shown as an implemented feature.

6. Implementation Details

6.1 Frontend

- API URL is read from VITE_API_URL, with the deployed Render API as the current default.
- Accepted browser MIME types are image/jpeg, image/png and image/webp.
- Client-side maximum size is 5 MB.
- The app checks /health on load and refreshes the status every 30 seconds.
- The selected image is previewed using a browser object URL, which is revoked when the selection is reset or replaced.
- The form submits multipart FormData with the field name `file` to /analyze.

6.2 Backend endpoints

Method / path	Purpose	Key behaviour
GET /	API root	Returns a running message
GET /health	Health check	Returns status=ok
POST /upload	Separate upload persistence	Validates, writes UUID-named image to uploads/ and stores metadata in SQLite
POST /analyze	Image classification	Validates image; runs local EfficientNet-B0; returns top-three JSON

6.3 Data persistence

The backend initialises a SQLite `cases` table with case identifier, original filename, image path, upload timestamp and status. This table is used by the separate `/upload` endpoint. The primary frontend analysis flow currently calls `/analyze` and does not use this persistence operation.

7. Dataset Preparation and Model Training

7.1 Dataset

The recorded seven-class training work used ISIC 2018 Task 3 with 10,015 labelled images. The class distribution is imbalanced, especially for NV compared with DF and VASC.

Class code	Category	Image count
MEL	Melanoma	1,113
NV	Melanocytic nevus	6,705
BCC	Basal cell carcinoma	514
AKIEC	Actinic keratoses / intraepithelial carcinoma	327
BKL	Benign keratosis-like lesion	1,099
DF	Dermatofibroma	115
VASC	Vascular lesion	142
Total	All classes	10,015

7.2 Group-aware split

Partition	Images	Lesion groups
Train	7,014	5,230
Validation	1,524	1,120
Test	1,477	1,120
Total	10,015	7,470

The recorded split used lesion-group-aware assignment with seed 20261020 and reported zero lesion-group leakage between partitions. The test partition was subsequently inspected during multiple experiment comparisons, so it should not be described as an untouched final benchmark.

7.3 Training experiments

Three EfficientNet-B0 experiments were completed. The strongest recorded run used frozen BatchNorm running statistics and a learning rate of 3e-5. Training used ImageNet-pretrained initialisation, weighted classification loss and image augmentation as recorded in the experiment workflow.

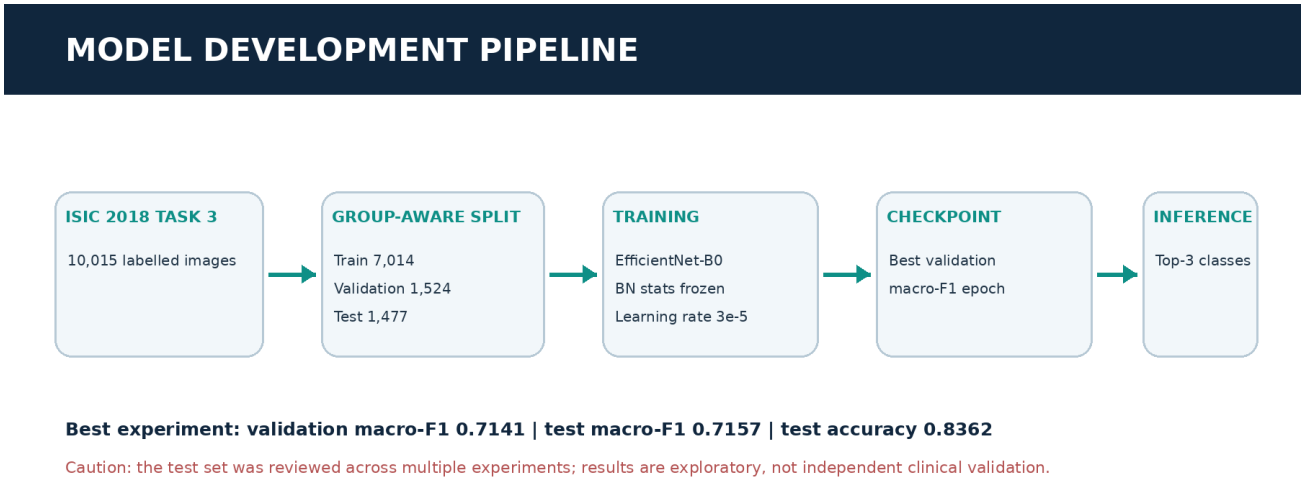


Figure 2. Recorded model-development workflow and best-run summary.

8. Experimental Results and Interpretation

Metric	Validation	Test (exploratory)
Accuracy	83.33%	83.62%
Balanced accuracy	71.75%	71.20%
Macro-F1	71.41%	71.57%
Loss	0.4512	0.4496

8.1 Test recall by class

Class	Recall
MEL - Melanoma	55.36%
NV - Melanocytic nevus	93.3%
BCC - Basal cell carcinoma	74.3%
AKIEC	63.8%
BKL	66.9%
DF - Dermatofibroma	50.0%
VASC - Vascular lesion	94.7%

The aggregate accuracy should not be interpreted alone. The class-wise recall demonstrates that performance differs by category; melanoma recall of 55.36% is an important limitation. The class imbalance also makes macro-averaged and class-wise metrics essential.

The recorded test metrics are exploratory because this test set was examined across several development experiments. A fresh independent test set is required for stronger generalisation evidence. No clinical performance claim is made.

9. Out-of-Domain Investigation

A candidate set of 437 Open Images object photographs was used to investigate whether the existing classifier would recognise non-skin inputs. The candidates covered unrelated object categories such as aircraft, banana, car, cat, chair, computer keyboard, dog, laptop, person and television. The sample was visually reviewed as unrelated to skin-lesion inputs, while individual dataset reuse rights remain under review.

Predicted class	Count
NV	322
MEL	38
VASC	37
BKL	24
BCC	12
DF	4
Total	437

All 437 unrelated candidates received one of the seven known classifier labels. This is expected behaviour for a closed-set classifier without a rejection mechanism and is the primary reason a separate input-suitability gate is required.

9.1 Exploratory confidence-threshold analysis

Threshold	Valid lesion validation accepted	OOD candidates rejected
0.50	93.04%	67.28%
0.60	84.97%	78.72%
0.70	77.69%	89.47%
0.80	69.49%	95.42%
0.90	57.94%	99.08%

At a 0.70 threshold, the exploratory analysis rejected 89.47% of OOD candidates but also rejected 22.31% of valid validation lesion images. This trade-off is not sufficient to establish a safe rejection gate. The OOD sample is limited and is not an independent medically validated benchmark.

10. API Design and Deployment

Component	Deployment / URL	Purpose
Frontend	https://derm-1.netlify.app	React + Vite user interface
Backend	https://dermai-api-r2in.onrender.com	FastAPI REST service
Swagger UI	https://dermai-api-r2in.onrender.com/docs	Interactive API documentation
Health check	https://dermai-api-r2in.onrender.com/health	Backend availability check

The frontend build uses Vite. The backend is started with Uvicorn. Production readiness depends on the model checkpoint being present and on dependency compatibility. Public demo services may have cold starts or temporary availability constraints; confirm connectivity before the faculty demonstration.

10.1 Local run commands

Backend: create/activate a Python virtual environment, install `backend/requirements.txt`, then run `uvicorn main:app --reload --port 8000` from the backend directory. Frontend: run `npm install` and `npm run dev` from the frontend directory. Set `VITE_API_URL=http://127.0.0.1:8000` to target the local API and ensure CORS allows the local frontend origin.

11. Testing, Security and Privacy

11.1 Implemented validation

- Frontend accepts JPEG, PNG and WEBP and enforces a 5 MB client-side limit.
- Backend independently checks supported MIME types and maximum file size.
- Pillow validates the actual image structure, verifies the file and fully decodes it before inference or persistence.
- The API returns structured HTTP errors for invalid, empty, oversized or unsupported uploads.
- The frontend displays API connectivity, loading and error feedback.

11.2 Manual verification cases for demonstration

Test case	Expected outcome
GET /health	HTTP 200 with status=ok
Valid JPG/PNG/WEBP <= 5 MB	Accepted by the inference endpoint if checkpoint and dependencies are available
Unsupported image type	HTTP 400
Empty or corrupt image	HTTP 400
Image > 5 MB	HTTP 413
Backend unavailable	Frontend indicates offline / connection error
Unrelated object image	May still receive a lesion label; reliable rejection is not implemented

The cases above are expected behaviours inferred from the reviewed source code. They are not a claim that a formal automated test suite has been executed.

11.3 Privacy and security considerations

- Use only images the uploader has permission to share. Avoid faces and identifying details.
- The separate /upload endpoint persists files and metadata locally; production must define retention, deletion and access-control policies.
- The reviewed code does not implement authentication or rate limiting.
- CORS uses an explicit origin allowlist; update it for the actual frontend domains.
- Do not store secrets, tokens, private images or local database files in source control.

- Use HTTPS in deployment and restrict access to logs and uploaded health-related images.

12. Limitations and Future Work

12.1 Current limitations

- No implemented non-skin rejection gate or validated image-quality classifier.
- Closed-set predictions are forced among seven known classes for unrelated inputs.
- Test results are exploratory and the test set was reused across model comparisons.
- Melanoma recall is 55.36% on the exploratory test evaluation.
- Softmax scores are not calibrated clinical probabilities.
- No independent external clinical validation or clinician sign-off is documented.
- Dataset and checkpoint licence/redistribution review is incomplete.
- Production security and data-governance features require further work.

12.2 Prioritised roadmap

Priority	Work package	Deliverable
P1	Image-suitability / non-skin rejection gate	Separate model and input policy; tested on permitted labelled data
P1	Independent evaluation	Untouched test set, class-wise recall, specificity, macro-F1 and calibration
P2	Model improvement	Address class imbalance and weak class-wise recall; assess skin-tone robustness
P2	Automated tests	Backend endpoint tests and frontend flow tests
P2	Privacy and reliability	Retention/deletion controls, authentication, rate limiting, monitoring
P3	Clinical and regulatory review	Expert review and jurisdiction-specific requirements before clinical claims
P3	Commercial readiness	Dataset/model rights and distribution permissions review

13. Conclusion

DermAI demonstrates an end-to-end academic prototype connecting a responsive React frontend, FastAPI backend, image validation pipeline and local EfficientNet-B0 classifier. The project includes dataset preparation, group-aware splitting, three training experiments, top-three inference and deployment of the web application.

The evaluation also reveals important limitations: the classifier can assign a known lesion label to unrelated images, class-wise recall is uneven, and the test set is not an untouched independent benchmark. Therefore, the current build should be presented as a research and engineering prototype rather than a diagnostic system. A separately developed rejection gate, stronger independent validation, privacy controls and expert review are the most important next steps.

Appendix A. Faculty Demonstration Script

1. Open https://derm-1.netlify.app and point out the research-preview label and upload workspace.
2. Show the supported formats (JPG, PNG, WEBP), 5 MB limit and selected-image preview.
3. Wait for the API status to show Connected.
4. Upload a permitted example image and click Analyze image.

5. Explain the top prediction, top-three alternatives and score. Clarify that the score is not a diagnosis or a medical probability.

6. Open Swagger UI and demonstrate /health and /analyze.

7. Present the model result table and mention the exploratory test-set limitation.

8. Explain the OOD investigation: all 437 unrelated candidates received one of seven labels; the rejection gate is future work.

Appendix B. References and Project Links

- Project repository: https://github.com/yadav32467/DreamAI
- Live frontend: https://derm-1.netlify.app
- Backend API: https://dermai-api-r2in.onrender.com
- FastAPI Swagger UI: https://dermai-api-r2in.onrender.com/docs
- ISIC Challenge data portal: https://challenge.isic-archive.com/data/
- FastAPI documentation: https://fastapi.tiangolo.com/
- PyTorch documentation: https://pytorch.org/docs/stable/index.html
- torchvision documentation: https://pytorch.org/vision/stable/index.html

This report was prepared from the currently inspected public repository files and recorded project experiment notes. Confirm deployment availability and fill in student/institution details before formal submission.